

# FOODGO

## Restaurant, Order & Delivery Management System

### DBMS COURSE PROJECT — FINAL REPORT

#### Team 5

A MySQL-based relational database for managing customers, addresses, restaurants, menus, orders, deliveries, payments and reviews.

### 1. Team and Project Information

FoodGo was developed as a Team 5 DBMS course project. The project brief identifies the system as a Restaurant, Order and Delivery Management System.

| S.No. | Student              | USN        |
|-------|----------------------|------------|
| 1     | Rehaan Saha          | AU25UG-046 |
| 2     | S Thanmayee          | AU25UG-049 |
| 3     | Samhita Bhuvanagiri  | AU25UG-050 |
| 4     | Sarang Siva Thilakan | AU25UG-054 |
| 5     | Shahiba Arshiya Banu | AU25UG-055 |
| 6     | Supriya R            | AU25UG-058 |

Project domain: Food delivery database

Primary DBMS: MySQL

Database name: foodgo

Core entities: 10 relational tables

### 2. Executive Summary

FoodGo models the core operations of a food-delivery platform in a relational database. The system connects customers, saved delivery addresses, restaurants, menu items, orders, order-item details, delivery partners, deliveries, payments and reviews.

The project follows the DBMS workflow from requirements analysis and ER modeling to relational design, integrity constraints, implementation, normalization analysis and SQL-based business analysis.

A dedicated read-only normalization validation script evaluates 1NF, 2NF and the relevant 3NF dependencies and verifies important data-consistency rules.

### 3. Problem Statement and Requirements

A food-delivery platform coordinates customers placing orders, restaurants preparing food and delivery partners fulfilling deliveries. Each order may involve menus, prices, addresses, payments, delivery assignments and customer ratings.

#### 3.1 Functional Scope

Manage customer accounts and saved delivery addresses.

Maintain restaurants and their menu items with prices.

Record orders and the items contained in each order.

Assign delivery partners and track pickup and delivery times.

Record payments and order status, including cancellations.

Capture customer reviews for restaurants and deliveries.

#### 3.2 Project Objectives

Translate a real-world data problem into a relational design.

Identify entities, attributes, keys, relationships, cardinalities and participation.

Map the ER model into relational tables.

Apply primary keys, foreign keys and integrity constraints.  
Identify functional dependencies and analyze normalization.  
Populate tables with realistic sample data.  
Write SQL queries that answer practical business questions.  
Document important design decisions and assumptions.

## 4. Final Repository Structure

The current repository structure shown in the submitted GitHub view is:

```
FoodGo---Food-Delivery-System/
├── ER diagram.drawio (1).png
├── diagrams/ → ER diagram.drawio (1).png
├── docs/ → README.md, Report
├── normalization/ → normalization_validation.sql
├── queries/ → queries.sql
└── schema/ → create_tables.sql
```

| Repository item                            | Purpose                                            |
|--------------------------------------------|----------------------------------------------------|
| ER diagram.drawio (1).png                  | ER diagram image at repository root                |
| diagrams/                                  | ER diagram image copy                              |
| docs/README.md                             | Project documentation                              |
| docs/Report                                | Project report                                     |
| normalization/normalization_validation.sql | Read-only normalization and consistency validation |
| queries/queries.sql                        | Business and analytical SQL queries                |
| schema/create_tables.sql                   | Database, table, key and constraint definitions    |

## 5. Technology Stack and Implementation

| Layer           | Technology                   |
|-----------------|------------------------------|
| RDBMS           | MySQL 8.x                    |
| Query Language  | SQL                          |
| Schema / DDL    | MySQL DDL                    |
| Data / DML      | SQL INSERT statements        |
| Database Design | Entity-Relationship modeling |
| Documentation   | Markdown / project report    |
| Repository      | GitHub                       |

The normalization execution evidence records MySQL version 8.0.36 with foreign-key checks enabled.

## 6. Relational Database Design

| Table             | Purpose                                   | Primary Key         |
|-------------------|-------------------------------------------|---------------------|
| customers         | Customer profiles and contact information | customer_id         |
| addresses         | Saved delivery addresses                  | address_id          |
| restaurants       | Partner restaurant information            | restaurant_id       |
| menu_items        | Food items offered by restaurants         | item_id             |
| delivery_partners | Delivery partner information              | partner_id          |
| orders            | Order-level information                   | order_id            |
| order_items       | Items within orders;<br>resolves M:N      | (order_id, item_id) |

|            |                                |             |
|------------|--------------------------------|-------------|
| deliveries | Delivery assignment and timing | delivery_id |
| payments   | Payment information            | payment_id  |
| reviews    | Ratings and comments           | review_id   |

OUTPUT RESULT after running create\_tables.sql and insert\_tables.sql.  
The following Tables were created under the Schema "foodgo"

|                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                              |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Table: addresses</b><br><br><b>Columns:</b><br><u>address_id</u> int AI PK<br><u>customer_id</u> int<br>line1 varchar(255)<br>area varchar(100)<br>city varchar(50)<br>pincode varchar(10)<br><br><b>Related Tables:</b><br>Target customers (customer_id → customer_id)<br>On Update CASCADE<br>On Delete CASCADE | <b>Table: customers</b><br><br><b>Columns:</b><br><u>customer_id</u> int AI PK<br>name varchar(100)<br>phone varchar(15)<br>email varchar(100)                                     | <b>Table: delivery_partners</b><br><br><b>Columns:</b><br><u>partner_id</u> int AI PK<br>name varchar(100)<br>phone varchar(15)<br>vehicle_type enum('bike','scooter','bicycle','car')                                                                                                       |
| <b>Table: menu_items</b><br><br><b>Columns:</b><br><u>item_id</u> int AI PK<br><u>restaurant_id</u> int<br>name varchar(150)<br>price decimal(8,2)<br>is_veg tinyint(1)                                                                                                                                               | <b>Table: deliveries</b><br><br><b>Columns:</b><br><u>delivery_id</u> int AI PK<br><u>order_id</u> int<br><u>partner_id</u> int<br>pickup_time datetime<br>delivered_time datetime | <b>Table: orders</b><br><br><b>Columns:</b><br><u>order_id</u> int AI PK<br><u>customer_id</u> int<br><u>restaurant_id</u> int<br><u>address_id</u> int<br>order_time datetime<br>status enum('placed','preparing','out_for_delivery','delivered','cancelled')<br>total_amount decimal(10,2) |
|                                                                                                                                                                                                                                                                                                                       | <b>Table: order_items</b><br><br><b>Columns:</b><br><u>order_id</u> int PK<br><u>item_id</u> int PK<br>quantity int<br>unit_price decimal(8,2)                                     | <b>Table: payments</b><br><br><b>Columns:</b><br><u>payment_id</u> int AI PK<br><u>order_id</u> int<br>amount decimal(10,2)<br>method enum('card','upi','cash','netbanking')<br>status enum('pending','completed','failed','refunded')                                                       |
|                                                                                                                                                                                                                                                                                                                       | <b>Table: restaurants</b><br><br><b>Columns:</b><br><u>restaurant_id</u> int AI PK<br>name varchar(150)<br>cuisine varchar(50)<br>city varchar(50)<br>is_active tinyint(1)         | <b>Table: reviews</b><br><br><b>Columns:</b><br><u>review_id</u> int AI PK<br><u>order_id</u> int<br>restaurant_rating int<br>delivery_rating int<br>comment text                                                                                                                            |

In version 2 of our create\_tables.sql file, we made some improvements without removing any existing functionality. The file increased from 198 lines to 284 lines.

First, we added three CHECK constraints to validate the format of important fields — customer phone numbers must contain 10 digits, address pincodes must contain 6 digits, and delivery partner phone numbers must contain 10 digits.

Next, we added two indexes on the orders table: one for order status and another for order time. These help improve the efficiency of queries that frequently search or sort orders using these columns.

We also added three triggers for the order\_items table. These triggers automatically recalculate the order's total\_amount whenever an order item is inserted, updated, or deleted. Before creating each trigger, we use DROP TRIGGER IF EXISTS so that the script can be safely executed again.

Finally, we added a changelog to document the modifications. The existing unique constraints were not removed; we only added trailing commas so the new CHECK constraints could follow them.

The important point is that these changes are backward-compatible. Our teammates' insert\_tables.sql, queries.sql, and normalization\_validation.sql files continue to work without modification and produce the same results. They only need to pull the updated create\_tables.sql when rebuilding the database from scratch.

## 7. Table-wise Schema Description

### Customers

Purpose: Customer profiles and contact information.

Attributes / keys: customer\_id PK; name; phone UNIQUE; email UNIQUE

### Addresses

Purpose: Saved delivery addresses.

Attributes / keys: address\_id PK; customer\_id FK; line1; area; city; pincode

### Restaurants

Purpose: Partner restaurant information.

Attributes / keys: restaurant\_id PK; name; cuisine; city; is\_active

### menu\_items

Purpose: Food items offered by restaurants.

Attributes / keys: item\_id PK; restaurant\_id FK; name; price; is\_veg

### delivery\_partners

Purpose: Delivery partner information.

Attributes / keys: partner\_id PK; name; phone UNIQUE; vehicle\_type

### Orders

Purpose: Order-level information.

Attributes / keys: order\_id PK; customer\_id FK; restaurant\_id FK; address\_id FK; order\_time; status; total\_amount

### order\_items

Purpose: Items within orders; resolves M:N.

Attributes / keys: order\_id FK; item\_id FK; quantity; unit\_price; composite PK (order\_id, item\_id)

### Deliveries

Purpose: Delivery assignment and timing.

Attributes / keys: delivery\_id PK; order\_id UNIQUE FK; partner\_id FK; pickup\_time; delivered\_time

### Payments

Purpose: Payment information.

Attributes / keys: payment\_id PK; order\_id UNIQUE FK; amount; method; status

### Reviews

Purpose: Ratings and comments.

Attributes / keys: review\_id PK; order\_id UNIQUE FK; restaurant\_rating; delivery\_rating; comment

## 8. Relationships and Cardinalities

| Relationship                | Cardinality | Explanation                                              |
|-----------------------------|-------------|----------------------------------------------------------|
| Customer → Address          | 1 : N       | A customer can maintain multiple saved addresses.        |
| Customer → Order            | 1 : N       | A customer can place multiple orders.                    |
| Restaurant → Menu Item      | 1 : N       | A restaurant offers multiple menu items.                 |
| Restaurant → Order          | 1 : N       | Each order belongs to one restaurant in the base design. |
| Order ↔ Menu Item           | M : N       | Implemented through order_items.                         |
| Order → Delivery            | 0 : 1       | Unique order_id in deliveries; at most one record.       |
| Delivery Partner → Delivery | 1 : N       | A partner can handle multiple deliveries.                |
| Order → Payment             | 0 : 1       | Unique order_id in payments; at most one record.         |

|                |       |                                                    |
|----------------|-------|----------------------------------------------------|
| Order → Review | 0 : 1 | Unique order_id in reviews;<br>at most one record. |
|----------------|-------|----------------------------------------------------|

## 9. Keys, Constraints and Data Integrity

### Primary and Composite Keys

Each core table has a primary key. order\_items uses the composite primary key (order\_id, item\_id).

### Foreign Keys

Foreign keys connect customers, addresses, restaurants, menu items, orders, delivery partners, deliveries, payments and reviews.

### UNIQUE

Unique constraints are used for customer phone/email, delivery partner phone and order\_id in deliveries, payments and reviews.

### CHECK

Prices, quantities, unit prices, totals and ratings are constrained to valid ranges.

### DEFAULT and ENUM

Default values such as order\_time/current timestamp and controlled ENUM domains are used.

### Referential Actions

CASCADE and RESTRICT actions are used to preserve referential integrity.

## 10. Normalization Analysis

The project contains a dedicated read-only normalization/normalization\_validation.sql file. The script is designed for demonstration and verification rather than database modification.

### 10.1 UNF

A conceptual order form may contain a repeating group of items. This is the starting normalization problem.

### 10.2 1NF

FoodGo represents each ordered item as a separate row in order\_items. Values are atomic and repeating groups are removed.

### 10.3 2NF

order\_items uses (order\_id, item\_id) as a composite key. The dependency (order\_id, item\_id) → quantity, unit\_price is demonstrated, while order-level and item-level facts are separated.

### 10.4 3NF

Under the stated business dependency address\_id → customer\_id, orders contains order\_id → address\_id → customer\_id. Because address\_id is not an orders superkey and customer\_id is non-prime, the current physical orders relation is not strict 3NF under that assumption.

### 10.5 Normalized Projection

N10 omits customer\_id from the order projection and N14 checks lossless reconstruction without changing the physical schema.

## 11. Normalization Example — Order 2

| Attribute    | Value                                               |
|--------------|-----------------------------------------------------|
| Order        | 2                                                   |
| Customer     | Sneha Nair (ID 5)                                   |
| Restaurant   | The Garden Continental (ID 8)                       |
| Address      | Address ID 6                                        |
| Item 1       | Penne Alfredo (ID 74), Qty 3, Unit Price 299        |
| Item 2       | Grilled Salmon Steak (ID 77), Qty 2, Unit Price 449 |
| Stored total | 1795                                                |

$3 \times 299 = 897$ ;  $2 \times 449 = 898$ ; Total = 1795. The validation output records stored total 1795, calculated total 1795 and difference 0.

## 12. Normalization and Consistency Validation Results

| ID  | Check                    | Result                                                               |
|-----|--------------------------|----------------------------------------------------------------------|
| N01 | Environment              | foodgo; MySQL 8.0.36;<br>foreign_key_checks = 1                      |
| N02 | Base tables              | 10 base tables identified                                            |
| N03 | Keys & constraints       | Primary and unique<br>constraints inspected                          |
| N04 | 1NF demonstration        | Order 2 represented as one<br>row per ordered item                   |
| N05 | 2NF order header         | Order-level facts separated                                          |
| N06 | 2NF menu facts           | Item-level facts separated                                           |
| N07 | Composite-key dependency | (order_id, item_id) → quantity,<br>unit_price                        |
| N08 | Address ownership        | Selected address ownership<br>checked                                |
| N09 | Ownership consistency    | 40 orders checked; 0 missing<br>addresses; 0 ownership<br>mismatches |
| N10 | 3NF projection           | Order projection shown<br>without customer_id                        |
| N11 | Address facts            | Address-level dependency<br>demonstrated                             |
| N12 | Customer facts           | Customer-level dependency<br>demonstrated                            |
| N13 | Restaurant facts         | Restaurant-level dependency<br>demonstrated                          |
| N14 | Lossless reconstruction  | 40 original; 40 reconstructed;<br>0 mismatched headers               |
| N15 | Price snapshot           | Current menu price compared<br>with historical unit_price            |
| N16 | Order total              | Order 2: stored 1795;<br>calculated 1795; difference 0               |
| N17 | All-order totals         | 40 orders; 0 without items; 0<br>total mismatches                    |
| N18 | Restaurant consistency   | 74 order-items; 0 missing<br>parents; 0 mismatches                   |
| N19 | Inventory                | 347 records across 10 tables                                         |

```

32
33 SELECT
34     'N01: Environment' AS demonstration,
35     DATABASE() AS database_name,
36     VERSION() AS mysql_version,
37     @@SESSION.foreign_key_checks AS foreign_key_checks;
38
39
40 -- N02. EXISTING BASE TABLES

```

```

44
45 SELECT
46     TABLE_NAME AS table_name
47 FROM information_schema.TABLES
48 WHERE TABLE_SCHEMA = 'foodgo'
49 AND TABLE_TYPE = 'BASE TABLE'
50 ORDER BY TABLE_NAME;
51
52 --

```

```

73
74 SELECT
75     tc.table_name AS table_name,
76     tc.constraint_name AS constraint_name,
77     tc.constraint_type AS constraint_type,
78     kcu.column_name AS key_column,
79     kcu.ordinal_position AS position_in_key,
80     col.is_nullable AS is_nullable
81 FROM information_schema.table_constraints AS tc
82 JOIN information_schema.key_column_usage AS kcu
83 ON tc.constraint_name = kcu.constraint_name

```

| Table_name        | constraint_name    | constraint_type | key_column    | position_in_key | is_nullable |
|-------------------|--------------------|-----------------|---------------|-----------------|-------------|
| addresses         | PRIMARY            | PRIMARY KEY     | address_id    | 1               | NO          |
| customers         | PRIMARY            | PRIMARY KEY     | customer_id   | 1               | NO          |
| customers         | uq_customers_email | UNIQUE          | email         | 1               | YES         |
| customers         | uq_customers_phone | UNIQUE          | phone         | 1               | NO          |
| delivers          | PRIMARY            | PRIMARY KEY     | delivery_id   | 1               | NO          |
| delivers          | uq_delivers_order  | UNIQUE          | order_id      | 1               | NO          |
| delivery_partners | PRIMARY            | PRIMARY KEY     | partner_id    | 1               | NO          |
| delivery_partners | uq_partners_phone  | UNIQUE          | phone         | 1               | NO          |
| menu_items        | PRIMARY            | PRIMARY KEY     | item_id       | 1               | NO          |
| order_items       | PRIMARY            | PRIMARY KEY     | order_id      | 1               | NO          |
| order_items       | PRIMARY            | PRIMARY KEY     | item_id       | 2               | NO          |
| orders            | PRIMARY            | PRIMARY KEY     | order_id      | 1               | NO          |
| payments          | PRIMARY            | PRIMARY KEY     | payment_id    | 1               | NO          |
| payments          | uq_payments_order  | UNIQUE          | order_id      | 1               | NO          |
| restaurants       | PRIMARY            | PRIMARY KEY     | restaurant_id | 1               | NO          |
| reviews           | PRIMARY            | PRIMARY KEY     | review_id     | 1               | NO          |
| reviews           | uq_reviews_order   | UNIQUE          | order_id      | 1               | NO          |

```

148 mi.name AS item_name,
149 oi.quantity,
150 oi.unit_price
151 FROM orders AS o
152 JOIN customers AS c
153 ON c.customer_id = o.customer_id
154 JOIN restaurants AS r
155 ON r.restaurant_id = o.restaurant_id
156 JOIN order_items AS oi
157 ON oi.order_id = o.order_id
158 JOIN menu_items AS mi
159 ON mi.item_id = oi.item_id
160 WHERE o.order_id = 2
161 ORDER BY oi.item_id

```

| order_id | item_id | order_time          | customer_id | customer_name | restaurant_id | restaurant_name        | item_name         | quantity | unit_price |
|----------|---------|---------------------|-------------|---------------|---------------|------------------------|-------------------|----------|------------|
| 2        | 74      | 2026-08-17 16:55:00 | 5           | Sneha Nair    | 8             | The Garden Continental | Veg Grill Platter | 3        | 299.00     |
| 2        | 77      | 2026-08-17 16:55:00 | 5           | Sneha Nair    | 8             | The Garden Continental | Steak             | 2        | 449.00     |

```

191 SELECT
192     order_id,
193     customer_id,
194     restaurant_id,
195     address_id,
196     order_time,
197     status,
198     total_amount
199 FROM orders
200 WHERE order_id = 2;
201

```

| order_id | customer_id | restaurant_id | address_id | order_time          | status    | total_amount |
|----------|-------------|---------------|------------|---------------------|-----------|--------------|
| 2        | 5           | 8             | 6          | 2026-08-17 16:55:00 | delivered | 1795.00      |

```

217 mi.name,
218 mi.price,
219 mi.is_veg
220 FROM menu_items AS mi
221 WHERE EXISTS (
222     SELECT 1
223     FROM order_items AS oi
224     WHERE oi.order_id = 2
225     AND oi.item_id = mi.item_id
226 )
227 ORDER BY mi.item_id;

```

| item_id | restaurant_id | name              | price  | is_veg |
|---------|---------------|-------------------|--------|--------|
| 74      | 8             | Veg Grill Platter | 299.00 | 1      |
| 77      | 8             | Steak             | 449.00 | 0      |

```

248
249 SELECT
250     order_id,
251     item_id,
252     quantity,
253     unit_price
254 FROM order_items
255 WHERE order_id = 2
256 ORDER BY item_id;
257
258 -- *****

```

| order_id | item_id | quantity | unit_price |
|----------|---------|----------|------------|
| 2        | 74      | 3        | 299.00     |
| 2        | 77      | 2        | 449.00     |

```

293
294 SELECT
295     o.order_id,
296     o.address_id,
297     o.customer_id AS stored_order_customer,
298     a.customer_id AS address_owner
299 FROM orders AS o
300 JOIN addresses AS a
301 ON a.address_id = o.address_id
302 WHERE o.address_id = 9
303 ORDER BY o.order_id;

```

| order_id | address_id | stored_order_customer | address_owner |
|----------|------------|-----------------------|---------------|
| 1        | 9          | 8                     | 8             |
| 4        | 9          | 8                     | 8             |

```

325 ) AS missing_address_orders,
326 COUNT(
327     CASE
328         WHEN a.address_id IS NOT NULL
329         AND o.customer_id <> a.customer_id
330         THEN 1
331     END
332 ) AS ownership_mismatches
333 FROM orders AS o
334 LEFT JOIN addresses AS a
335 ON a.address_id = o.address_id;

```

| checked_orders | missing_address_orders | ownership_mismatches |
|----------------|------------------------|----------------------|
| 40             | 0                      | 0                    |

```

352 -- *****
353
354 SELECT
355     order_id,
356     restaurant_id,
357     address_id,
358     order_time,
359     status,
360     total_amount
361 FROM orders
362 WHERE order_id = 2;

```

| order_id | restaurant_id | address_id | order_time          | status    | total_amount |
|----------|---------------|------------|---------------------|-----------|--------------|
| 2        | 8             | 6          | 2026-08-17 16:55:00 | delivered | 1795.00      |

```

418 SELECT
419     r.restaurant_id,
420     r.name,
421     r.cuisine,
422     r.city,
423     r.is_active
424 FROM restaurants AS r
425 JOIN orders AS o
426     ON o.restaurant_id = r.restaurant_id
427 WHERE o.order_id = 2;

```

| restaurant_id | name                   | cuisine     | city      | is_active |
|---------------|------------------------|-------------|-----------|-----------|
| 8             | The Garden Continental | Continental | Bengaluru | 0         |

```

375 SELECT
376     a.address_id,
377     a.customer_id,
378     a.line1,
379     a.area,
380     a.city,
381     a.pincode
382 FROM addresses AS a
383 JOIN orders AS o
384     ON o.address_id = a.address_id
385 WHERE o.order_id = 2;

```

| address_id | customer_id | line1                 | area                   | city      | pincode |
|------------|-------------|-----------------------|------------------------|-----------|---------|
| 6          | 5           | No. 101, Airport Road | Banashankari 3rd Stage | Bengaluru | 560085  |

```

399 SELECT
400     c.customer_id,
401     c.name
402 FROM orders AS o
403 JOIN addresses AS a
404     ON a.address_id = o.address_id
405 JOIN customers AS c
406     ON c.customer_id = a.customer_id
407 WHERE o.order_id = 2;

```

| customer_id | name       |
|-------------|------------|
| 5           | Sneha Nair |

```

466 WITH normalized_order_projection AS (
467     SELECT
468         order_id,
469         restaurant_id,
470         address_id,
471         order_time,
472         status,
473         total_amount
474     FROM orders
475 ),
476 reconstructed_orders AS (
477     SELECT
478         n.order_id,
479         a.customer_id,
480         n.restaurant_id,
481         n.address_id,
482         n.order_time,
483         n.status,
484         n.total_amount
485     FROM normalized_order_projection AS n
486 )

```

| original_orders | reconstructed_orders | mismatched_headers |
|-----------------|----------------------|--------------------|
| 40              | 40                   | 0                  |

```

547 SELECT
548     oi.order_id,
549     oi.item_id,
550     mi.name AS item_name,
551     mi.price AS current_menu_price,
552     oi.unit_price AS historical_unit_price,
553     oi.quantity,
554     oi.quantity * oi.unit_price AS line_amount
555 FROM order_items AS oi
556 JOIN menu_items AS mi
557     ON mi.item_id = oi.item_id
558 WHERE oi.order_id = 2
559 ORDER BY oi.item_id;

```

| order_id | item_id | item_name         | current_menu_price | historical_unit_price | quantity | line_amount |
|----------|---------|-------------------|--------------------|-----------------------|----------|-------------|
| 2        | 74      | Veg Grill Platter | 299.00             | 299.00                | 3        | 897.00      |
| 2        | 77      | Steak             | 449.00             | 449.00                | 2        | 898.00      |

```

574 --
575 -- *****
576
577 SELECT
578     o.order_id,
579     o.total_amount AS stored_total,
580     SUM(oi.quantity * oi.unit_price) AS calculated_total,
581     o.total_amount
582     - SUM(oi.quantity * oi.unit_price) AS difference
583 FROM orders AS o
584 JOIN order_items AS oi
585     ON oi.order_id = o.order_id
586 WHERE o.order_id = 2
587 GROUP BY
588     o.order_id,
589     o.total_amount;

```

| order_id | stored_total | calculated_total | difference |
|----------|--------------|------------------|------------|
| 2        | 1795.00      | 1795.00          | 0.00       |

```

608 SELECT
609     o.order_id,
610     (oi.quantity * oi.unit_price) AS calculated_total,
611     FROM order_items
612     GROUP BY order_id
613 ) AS checked_orders,
614
615 SELECT
616     CASE
617         WHEN o.order_id IS NULL THEN 1
618         WHEN o.order_id IS NOT NULL
619             AND o.restaurant_id <> mi.restaurant_id
620             THEN 1
621         ELSE 0
622     END AS restaurant_mismatches,
623     FROM order_items AS oi
624     LEFT JOIN orders AS o
625         ON o.order_id = oi.order_id
626     LEFT JOIN menu_items AS mi
627         ON mi.item_id = oi.item_id

```

| checked_orders | orders_without_items | mismatched_totals |
|----------------|----------------------|-------------------|
| 40             | 0                    | 0                 |

```

655 SELECT
656     WHEN o.order_id IS NULL
657     OR mi.item_id IS NULL
658     THEN 1
659     ) AS missing_parent_lines,
660
661 COUNT(
662     CASE
663         WHEN o.order_id IS NOT NULL
664         AND mi.item_id IS NOT NULL
665         AND o.restaurant_id <> mi.restaurant_id
666         THEN 1
667         ELSE 0
668     ) AS restaurant_mismatches,
669
670 FROM order_items AS oi
671 LEFT JOIN orders AS o
672     ON o.order_id = oi.order_id
673 LEFT JOIN menu_items AS mi
674     ON mi.item_id = oi.item_id

```

| checked_order_items | missing_parent_lines | restaurant_mismatches |
|---------------------|----------------------|-----------------------|
| 74                  | 0                    | 0                     |

```

719 UNION ALL
720     SELECT 'deliveries', COUNT(*) FROM deliveries
721
722 UNION ALL
723     SELECT 'payments', COUNT(*) FROM payments
724
725 UNION ALL
726     SELECT 'reviews', COUNT(*) FROM reviews
727
728 SELECT
729     table_name,
730     row_count
731 FROM row_counts
732
733 UNION ALL

```

| table_name        | row_count |
|-------------------|-----------|
| customers         | 25        |
| addresses         | 30        |
| restaurants       | 8         |
| menu_items        | 80        |
| delivery_partners | 30        |
| orders            | 40        |
| order_items       | 74        |
| deliveries        | 32        |
| payments          | 36        |
| reviews           | 12        |
| TOTAL             | 347       |

## 13. Database Inventory

| Table | Records |
|-------|---------|
|-------|---------|

|                   |     |
|-------------------|-----|
| customers         | 15  |
| addresses         | 20  |
| restaurants       | 10  |
| menu_items        | 80  |
| delivery_partners | 12  |
| orders            | 40  |
| order_items       | 74  |
| deliveries        | 36  |
| payments          | 34  |
| reviews           | 26  |
| TOTAL             | 347 |

The N19 validation output reports 347 records across ten base tables. The submitted sample-data source also documents a 347-row dataset, but its per-table distribution differs from the N19 validation snapshot; therefore this report uses N19 as the validation evidence.

## 14. SQL Business Analysis

The submitted queries/queries.sql addresses the seven core questions from the project brief and adds five further domain questions, for twelve questions in total.

| ID  | Business Question                                              | Main SQL Concepts                                     |
|-----|----------------------------------------------------------------|-------------------------------------------------------|
| Q1  | What is the most ordered food item?                            | JOIN, SUM(quantity), GROUP BY                         |
| Q2  | Which restaurant has the highest revenue?                      | Multi-table JOIN, SUM, ranking                        |
| Q3  | What is the average order value?                               | CTE/subquery, AVG                                     |
| Q4  | Who are the top customers by spending?                         | Aggregation, LIMIT                                    |
| Q5  | What is the average delivery time?                             | Time-difference functions, AVG                        |
| Q6  | Which restaurants have high ratings?                           | AVG, HAVING                                           |
| Q7  | How many orders were cancelled?                                | COUNT, CASE, filtering                                |
| Q8  | What are the peak ordering hours?                              | HOUR(), GROUP BY                                      |
| Q9  | How does delivery-partner performance compare?                 | JOIN, AVG, aggregation/ranking                        |
| Q10 | What is the revenue by cuisine?                                | Multi-level GROUP BY                                  |
| Q11 | Which customers order from multiple restaurants?               | COUNT(DISTINCT ...), HAVING                           |
| Q12 | What revenue is associated with cancelled orders?              | Conditional aggregation                               |
| Q13 | What are the most popular payment methods?                     | Aggregate functions + GROUP BY + ORDER BY             |
| Q14 | How do average restaurant ratings compare to delivery ratings? | Aggregate functions + NULL handling + WHERE filtering |
| Q15 | Which delivery vehicle type handles the most orders?           | JOIN + GROUP BY + COUNT + ORDER BY                    |

normalization\_validation insert\_tables queries create\_tables

```
194 AND delivery_rating IS NOT NULL; -- Filters out incomplete reviews
195
196 -----
197 -- Q15 [Extra]: Which delivery vehicle type handles the most orders?
198 -----
199 • SELECT
200     dp.vehicle_type, -- Type of vehicle used by partner
201     COUNT(d.delivery_id) AS total_deliveries -- Counts total deliveries
202 FROM delivery_partners dp
203 JOIN deliveries d ON dp.partner_id = d.partner_id
204 GROUP BY dp.vehicle_type -- Groups delivery counts by vehicle type
205 ORDER BY total_deliveries DESC; -- Sorts from highest volume
```

Result Grid

| vehicle_type | total_deliveries |
|--------------|------------------|
| bike         | 19               |
| scooter      | 11               |
| bicycle      | 2                |

normalization\_validation insert\_tables queries create\_tables

```
175
176 -- Q13 [Extra]: What are the most popular payment methods?
177 -----
178 SELECT
179     method, -- Payment method type
180     COUNT(*) AS total_transactions, -- Counts how many transactions
181     SUM(amount) AS total_amount_processed -- Sums up total amount
182 FROM payments
183 GROUP BY method -- Groups the counts by method
184 ORDER BY total_transactions DESC; -- Sorts from most frequent
```

Result Grid

| method     | total_transactions | total_amount_processed |
|------------|--------------------|------------------------|
| upi        | 13                 | 5517.00                |
| card       | 12                 | 5910.00                |
| netbanking | 7                  | 4300.00                |
| cash       | 4                  | 3396.00                |

normalization\_validation insert\_tables queries create\_tables

```
184 ORDER BY total_transactions DESC; -- Sorts from most frequently used to least
185
186 -----
187 -- Q14 [Extra]: How do average restaurant ratings compare to delivery ratings?
188 -----
189 SELECT
190     ROUND(AVG(restaurant_rating), 2) AS overall_avg_restaurant_score, -- Platform
191     ROUND(AVG(delivery_rating), 2) AS overall_avg_delivery_score -- Platform
192 FROM reviews
193 WHERE restaurant_rating IS NOT NULL
194 AND delivery_rating IS NOT NULL; -- Filters out incomplete reviews to ensure accuracy
```

Result Grid

| overall_avg_restaurant_score | overall_avg_delivery_score |
|------------------------------|----------------------------|
| 4.00                         | 4.00                       |

Query 1 insert\_tables normalization\_validation create\_tables queries

```
164 c.name AS customer_name,
165 COUNT(DISTINCT o.restaurant_id) AS unique_restaurants_tried
166 FROM customers c
167 JOIN orders o ON c.customer_id = o.customer_id
168 GROUP BY c.customer_id, c.name
169 HAVING COUNT(DISTINCT o.restaurant_id) > 1
170 ORDER BY unique_restaurants_tried DESC;
171
172 -----
173 -- Q12 [Extra]: Total revenue lost due to order cancellations
174 -- Concepts: Conditional aggregation, JOIN
175 -----
176 • SELECT
177     COUNT(o.order_id) AS cancelled_order_count,
178     SUM(o.total_amount) AS estimated_revenue_lost
179 FROM orders o
180 WHERE o.status = 'cancelled';
```

Result Grid

| cancelled_order_count | estimated_revenue_lost |
|-----------------------|------------------------|
| 4                     | 2139.00                |

Query 1 insert\_tables normalization\_validation create\_tables queries

```
155 GROUP BY r.cuisine
156 ORDER BY cuisine_revenue DESC;
157
158 -----
159 -- Q11 [Extra]: Customers ordering from multiple different restaurants
160 -- Concepts: HAVING COUNT(DISTINCT ...), JOIN
161 -----
162 • SELECT
163     c.customer_id,
164     c.name AS customer_name,
165     COUNT(DISTINCT o.restaurant_id) AS unique_restaurants_tried
166 FROM customers c
167 JOIN orders o ON c.customer_id = o.customer_id
168 GROUP BY c.customer_id, c.name
169 HAVING COUNT(DISTINCT o.restaurant_id) > 1
170 ORDER BY unique_restaurants_tried DESC;
171
172 -----
```

Result Grid

| customer_id | customer_name    | unique_restaurants_tried |
|-------------|------------------|--------------------------|
| 22          | Deepak Verma     | 4                        |
| 2           | Rohan Sharma     | 3                        |
| 7           | Divya Kumar      | 3                        |
| 9           | Ananya Bhat      | 3                        |
| 23          | Nandini Krishnan | 3                        |
| 24          | Harsh Raghavan   | 3                        |
| 4           | Karthik Reddy    | 2                        |
| 8           | Vikram Shetty    | 2                        |
| 14          | Aditi Mehta      | 2                        |

Query 1 insert\_tables normalization\_validation create\_tables queries

```
141
142 -- Execute the selected portion of the script or everything, if there is no selection
143 -- Q10 [Extra]: Cuisine-wise revenue breakdown
144 -- Concepts: Multi-level GROUP BY, SUM
145 -----
146 • SELECT
147     r.cuisine,
148     COUNT(DISTINCT r.restaurant_id) AS active_restaurants,
149     COUNT(o.order_id) AS total_orders_delivered,
150     SUM(oi.quantity * oi.unit_price) AS cuisine_revenue
151 FROM restaurants r
152 JOIN orders o ON r.restaurant_id = o.restaurant_id
153 JOIN order_items oi ON o.order_id = oi.order_id
154 WHERE o.status = 'delivered'
155 GROUP BY r.cuisine
156 ORDER BY cuisine_revenue DESC;
157
158 -----
```

Result Grid

| cuisine      | active_restaurants | total_orders_delivered | cuisine_revenue |
|--------------|--------------------|------------------------|-----------------|
| Italian      | 10                 | 10                     | 3764.00         |
| Biryani      | 1                  | 7                      | 2570.00         |
| North Indian | 1                  | 10                     | 2517.00         |
| Continental  | 1                  | 3                      | 2244.00         |
| Desserts     | 1                  | 8                      | 2107.00         |
| Chinese      | 1                  | 4                      | 1731.00         |
| Fast Food    | 1                  | 3                      | 656.00          |
| South Indian | 1                  | 2                      | 168.00          |

Query 1 insert\_tables normalization\_validation create\_tables queries

```
126 -- Q9 [Extra]: Delivery partner performance & speed ranking
127 -- Concepts: JOIN, AVG delivery duration, ranking
128 -----
129 • SELECT
130     dp.partner_id,
131     dp.name AS partner_name,
132     dp.vehicle_type,
133     COUNT(d.delivery_id) AS completed_deliveries,
134     ROUND(AVG(TIMESTAMPDIFF(MINUTE, d.pickup_time, d.delivered_time)), 2) AS avg_delivery_minutes
135 FROM delivery_partners dp
136 JOIN deliveries d ON dp.partner_id = d.partner_id
137 JOIN orders o ON d.order_id = o.order_id
138 WHERE o.status = 'delivered' AND d.delivered_time IS NOT NULL
139 GROUP BY dp.partner_id, dp.name, dp.vehicle_type
140 ORDER BY avg_delivery_minutes ASC;
141
142 -----
```

Result Grid

| partner_id | partner_name | vehicle_type | completed_deliveries | avg_delivery_minutes |
|------------|--------------|--------------|----------------------|----------------------|
| 2          | Ravi Kumar   | bike         | 2                    | 15.50                |
| 4          | Sarosh       | scooter      | 2                    | 25.00                |
| 8          | Prakash      | bicycle      | 1                    | 25.00                |
| 1          | Manjunath    | bike         | 2                    | 29.50                |
| 3          | Faizan       | bike         | 3                    | 29.67                |
| 5          | Gopal        | bike         | 1                    | 30.00                |
| 6          | Ishai        | bike         | 3                    | 30.00                |
| 7          | Naveen       | scooter      | 7                    | 35.29                |
| 16         | Shruti       | bike         | 3                    | 44.00                |

Query 1 insert\_tables normalization\_validation create\_tables queries

```
111 FROM orders
112 WHERE status = 'cancelled';
113
114 -----
115 -- Q8 [Extra]: What are the peak ordering hours?
116 -- Concepts: HOUR(), GROUP BY
117 -----
118 • SELECT
119     HOUR(order_time) AS order_hour,
120     COUNT(*) AS total_orders
121 FROM orders
122 GROUP BY HOUR(order_time)
123 ORDER BY total_orders DESC;
124
125 -----
```

Result Grid

| order_hour | total_orders |
|------------|--------------|
| 8          | 5            |
| 18         | 4            |
| 14         | 4            |
| 15         | 3            |
| 22         | 3            |
| 30         | 3            |
| 19         | 3            |
| 11         | 3            |
| 21         | 3            |
| 9          | 3            |
| 17         | 2            |
| 20         | 2            |



Q8 groups orders by extracted order hour.  
 Q9 compares delivery partners using average delivery duration.  
 Q12 uses conditional aggregation for cancelled-order revenue.  
 Q13 use Aggregate functions + GROUP BY + ORDER BY  
 Q14 uses Aggregate functions + NULL handling + WHERE filtering  
 Q15 uses JOIN + GROUP BY + COUNT + ORDER BY

## 16. Key Design Decisions and Rationale

### Historical Price Snapshot

menu\_items.price is the current catalog price; order\_items.unit\_price is the historical purchase-time price.

### Composite Key

(order\_id, item\_id) uniquely identifies an item within an order and resolves the order-menu many-to-many relationship.

### At-Most-One Related Record

Unique order\_id in deliveries, payments and reviews permits at most one record of each type per order.

### Stored Order Total

orders.total\_amount is stored and checked against SUM(quantity × unit\_price).

### Read-Only Normalization

The validation script documents dependencies and verifies data without changing the submitted schema.

## 17. Assumptions, Limitations and Future Improvements

An address belongs to one customer, which is the key dependency used in the 3NF analysis.  
 Each menu item belongs to one restaurant.  
 order\_items. unit\_price preserves historical purchase pricing.  
 The current schema does not implement live payment gateways, GPS tracking, authentication or real-time dispatch.  
 A future redesign could remove orders.customer\_id and derive the customer through addresses, but existing queries and application logic would need review.

## 19. Requirement-to-Submission Mapping

| Course Requirement    | FoodGo Evidence                                                              |
|-----------------------|------------------------------------------------------------------------------|
| Requirement analysis  | Scope, objectives and assumptions                                            |
| ER diagram            | ER diagram image at root and diagrams/                                       |
| Logical design        | schema/create_tables.sql and schema documentation                            |
| Integrity constraints | PK, FK, UNIQUE, NOT NULL, CHECK, DEFAULT, ENUM, referential actions          |
| Normalization         | normalization/normalization_validation.sql and N01-N19                       |
| Implementation        | MySQL foodgo database                                                        |
| Sample data           | Submitted sample-data SQL source                                             |
| SQL queries           | queries/queries.sql with 12 business questions                               |
| Design rationale      | Price snapshot, composite key, total verification and normalization analysis |
| GitHub organization   | schema/, queries/, normalization/, diagrams/, docs/ and root ER image        |
| Final documentation   | docs/README.md and docs/Report                                               |

## 20. Conclusion

FoodGo demonstrates the complete DBMS project lifecycle for a restaurant and food-delivery domain: requirements analysis, ER modeling, relational schema design, integrity constraints, data population, normalization analysis, SQL business queries and documentation.

The ten-table relational model provides structured relationships among customers, addresses, restaurants, menu items, orders, order items, delivery partners, deliveries, payments and reviews.

The normalization analysis is deliberately transparent: it demonstrates 1NF and 2NF, evaluates the 3NF dependency in orders, verifies lossless reconstruction and checks price, total and cross-table consistency without changing the submitted physical schema.

This report consolidates the project requirements, repository structure, database design, normalization evidence, SQL analysis, design decisions and reproduction steps into one final-submission document.